

Efficient Simulation of Stochastic Reaction Networks

Michaelis–Menten SRN

Ephrem Borie

Luigi Donnarumma

Giovanni Pomati

November 2025

Abstract

We study a stochastic reaction network (SRN) describing Michaelis–Menten enzyme kinetics and compare exact and approximate simulation schemes. We implement the Stochastic Simulation Algorithm (SSA), explicit and implicit tau-leaping, and the Chemical Langevin Equation (CLE) discretized by Euler–Maruyama. Selected code excerpts are reported in the Appendix. For each method we discuss qualitative trajectory behavior and provide Monte Carlo estimates of the quantity of interest, including confidence intervals. Finally, we consider a rare-event probability and illustrate variance reduction using Importance Sampling.

1 Stochastic reaction network model

We consider four species: S_1 , S_2 , S_3 , and S_4 . The reaction channels are



Let $X(t) = (X_1(t), X_2(t), X_3(t), X_4(t)) \in \mathbb{Z}_+^4$ denote the (integer) molecule counts at time t . Each reaction R_j is defined by a stoichiometric vector ν_j and a propensity function $a_j(\cdot)$.

1.1 Stoichiometry and propensities

In the species order (S_1, S_2, S_3, S_4) , the stoichiometric vectors are

$$\nu_1 = (-1, -1, 1, 0), \quad \nu_2 = (1, 1, -1, 0), \quad \nu_3 = (0, 1, -1, 1).$$

Under stochastic mass-action kinetics, the propensities are

$$\begin{aligned} a_1(x) &= c_1 \left(\frac{x_1!}{(x_1-1)!} \cdot \mathbb{1}_{x_1 \geq 1} \cdot \frac{x_2!}{(x_2-1)!} \cdot \mathbb{1}_{x_2 \geq 1} \cdot \mathbb{1}_{x_3 \geq 0} \cdot \mathbb{1}_{x_4 \geq 0} \right) = \\ &= c_1 (x_1 \cdot \mathbb{1}_{x_1 \geq 1} \cdot x_2 \cdot \mathbb{1}_{x_2 \geq 1} \cdot \mathbb{1}_{x_3 \geq 0} \cdot \mathbb{1}_{x_4 \geq 0}) \end{aligned} \tag{2}$$

$$\begin{aligned} a_2(x) &= c_2 (\mathbb{1}_{x_1 \geq 0} \cdot \mathbb{1}_{x_2 \geq 0} \cdot \frac{x_3!}{(x_3-1)!} \cdot \mathbb{1}_{x_3 \geq 1} \cdot \mathbb{1}_{x_4 \geq 0}) = \\ &= c_2 (\mathbb{1}_{x_1 \geq 0} \cdot \mathbb{1}_{x_2 \geq 0} \cdot x_3 \cdot \mathbb{1}_{x_3 \geq 1} \cdot \mathbb{1}_{x_4 \geq 0}) \end{aligned} \tag{3}$$

$$\begin{aligned} a_3(x) &= c_3 (\mathbb{1}_{x_1 \geq 0} \cdot \mathbb{1}_{x_2 \geq 0} \cdot \frac{x_3!}{(x_3-1)!} \cdot \mathbb{1}_{x_3 \geq 1} \cdot \mathbb{1}_{x_4 \geq 0}) = \\ &= c_3 (\mathbb{1}_{x_1 \geq 0} \cdot \mathbb{1}_{x_2 \geq 0} \cdot x_3 \cdot \mathbb{1}_{x_3 \geq 1} \cdot \mathbb{1}_{x_4 \geq 0}) \end{aligned} \tag{4}$$

(The process is constrained to remain in $\mathbb{Z}_+^4$.)

2 Exact simulation: Stochastic Simulation Algorithm (SSA)

The SSA simulates the continuous-time Markov chain exactly by sampling: (i) the time to the next reaction, which is exponentially distributed with rate $a_0(x) = \sum_{j=1}^3 a_j(x)$, and (ii) the index of the next reaction, sampled with probabilities $a_j(x)/a_0(x)$. After a reaction j fires, the state is updated as $X \leftarrow X + \nu_j$ and time is advanced accordingly.

2.1 Qualitative behavior of SSA trajectories

We used the parameters

$$c_1 = 0.0017, \quad c_2 = 10^{-4}, \quad c_3 = 0.1,$$

with initial condition

$$X_1(0) = 312, \quad X_2(0) = 125, \quad X_3(0) = 0, \quad X_4(0) = 0,$$

and simulated until $T = 50$. The specie S_1 decreases while the product S_4 increases, indicating conversion of S_1 into product S_4 . The S_2 and S_3 exhibit an approximately inverse relationship: rapid early binding forms S_3 , followed by a longer regime where it persists, and finally a decay as S_1 becomes scarce and S_2 is released. This overall behavior is consistent with the Michaelis-Menten system 1. Indeed, at the very beginning, the propensity of reaction channel 1 is the largest, so S_1 and S_2 turn in S_3 . At this point, the propensity of channels 2 and 3 start to be relevant, and so specie S_3 turns in S_1 and S_2 , or in S_2 and S_4 . The second case is more relevant, since S_4 is only a product of the chemical system, the quantity of the reagents decreases (because mass is conserved). In particular, the more S_3 react through channel 3, the less S_1 is available for reaction 1 and so its propensity decreases, making the consumption of S_1 slower. We expect that when the system reaches equilibrium, all the mass is distributed between S_2 and S_4 , since the products of reaction 3 are the only ones that cannot react again by themselves. Intuitively, S_1 is turning into S_4 with the help of S_2 , that makes the "auxiliary" reactions possible (the ones that produce and consume S_3). The trend described here is clearer in figure 1, where equilibrium point is almost reached and all the evolution is visible.

2.2 Monte Carlo estimation of $\mathbb{E}[X_4(T)]$ with SSA

We estimate the quantity of interest

$$Z = \mathbb{E}[X_4(T)]$$

by the Monte Carlo estimator

$$\hat{Z}_N = \frac{1}{N} \sum_{k=1}^N X_4^{(k)}(T),$$

where $X_4^{(k)}(T)$ is the terminal value of the k -th independent SSA sample path. For a confidence level $1 - \alpha$, we use the standard asymptotic confidence interval

$$\hat{Z}_N \pm z_{1-\alpha/2} \frac{\hat{\sigma}_N}{\sqrt{N}},$$

where $\hat{\sigma}_N^2$ is the sample variance and $z_{1-\alpha/2}$ is the corresponding Normal quantile. We fix the threshold ε for the half-width of this confidence interval and increase the number of simulations to achieve it. In particular, since the quantity we are trying to estimate is an integer, it is reasonable to pick $\varepsilon = 1$ to be accurate enough. With significance level $\alpha = 0.05$, our SSA simulations yielded

$$\hat{Z}_N \approx 296.818, \quad N = 55,$$

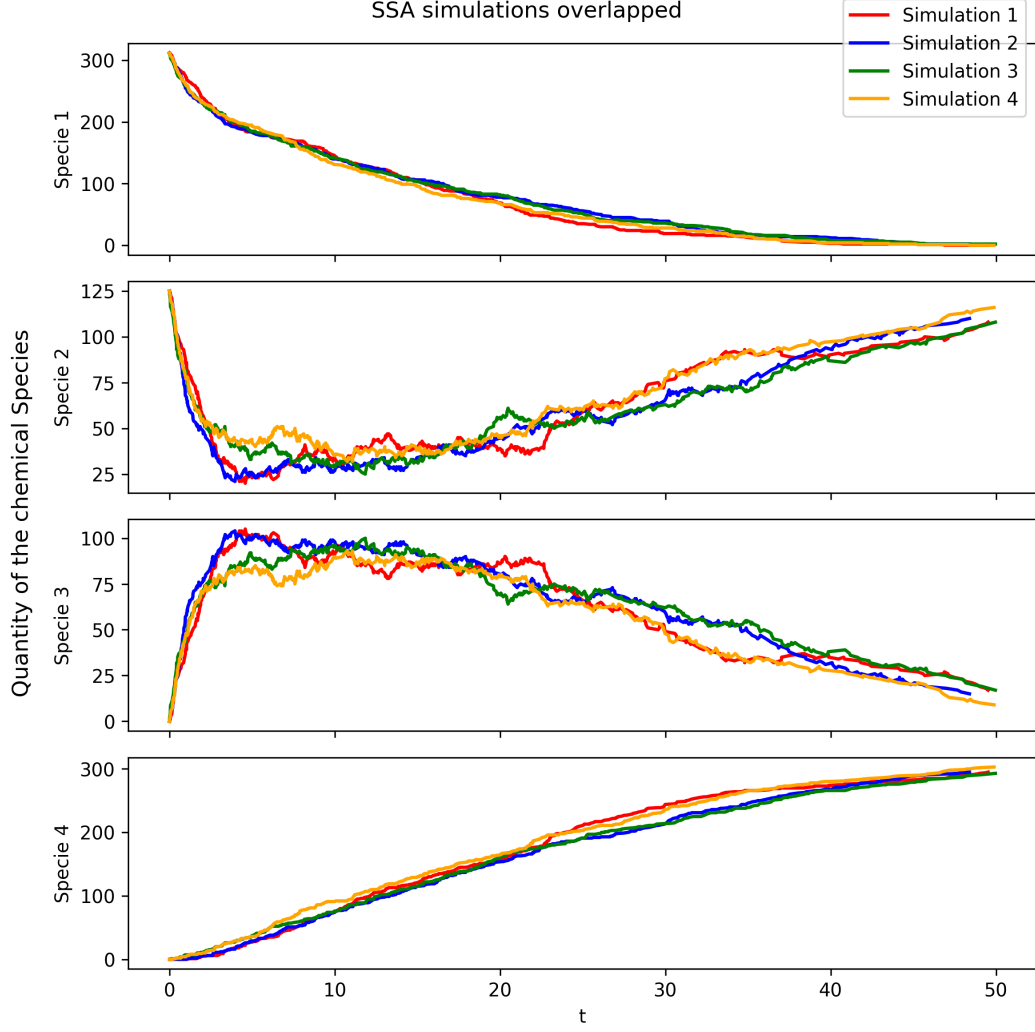


Figure 1: SSA: example trajectories of $X_i(t)$ for $i = 1, \dots, 4$ up to $T = 50$.

with confidence interval

$$CI = (295.889, 297.746),$$

and estimated variance $\widehat{\text{Var}}(\widehat{Z}_N) \approx 0.2243$. Overall, the estimator exhibited low variability and a narrow confidence interval consistently with the threshold fixed. In figure 2 we show the trend of the estimator with its confidence interval until it reaches the desired tolerance. While, in figure 3, we show the trend of the estimator for larger values of N . The last value will be taken as a reference for comparison in the next experiments with approximated algorithms.

2.3 Rare event probability and Importance Sampling

We consider the rare event

$$\{X_3(T) > 22\}, \quad g(X(T)) = \chi_{\{X_3(T) > 22\}}, \quad Z = \mathbb{E}[g(X(T))],$$

with different parameters than before

$$c_1 = 0.001, \quad c_2 = 0.005, \quad c_3 = 0.01,$$

and different initial state

$$X_1(0) = 100, \quad X_2(0) = 100, \quad X_3(0) = 0, \quad X_4(0) = 0,$$

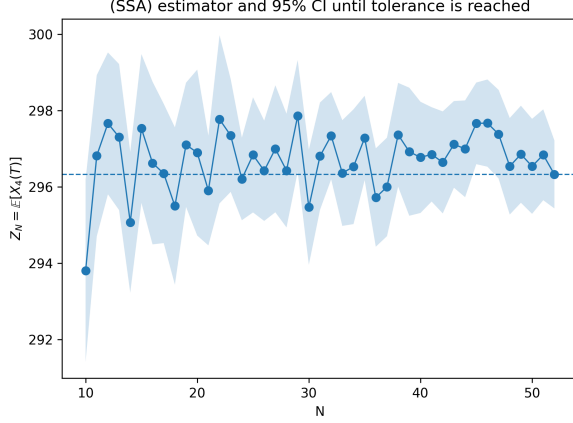


Figure 2: Monte Carlo estimation of $\mathbb{E}[X_4(T)]$ and its C.I. through SSA vs N , until desired tolerance is reached.

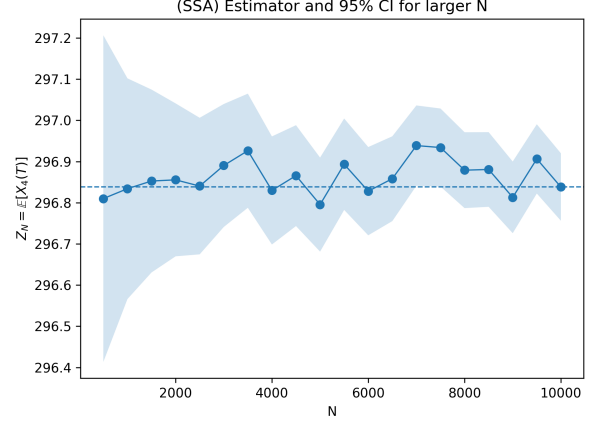


Figure 3: Monte Carlo estimation of $\mathbb{E}[X_4(T)]$ and its C.I. through SSA vs larger N ,

and final time $T = 1$.

2.3.1 Crude Monte Carlo (CMC)

Using crude Monte Carlo, the event was essentially unobserved for sample sizes up to 10^4 , producing an empirical estimate 0 and hence an (incorrectly) vanishing variance estimate. Only at $N = 10^5$ we observed the event for just 3 times, obtaining

$$\hat{Z}_{10^5} = 3 \times 10^{-5}, \quad \widehat{\text{Var}}(\hat{Z}_{10^5}) = 2.999 \times 10^{-10}.$$

This confirms that $\{X_3(T) > 22\}$ is a rare event under the nominal dynamics.

2.3.2 Importance Sampling (IS) and variance reduction

To reduce variance, we modify the rate constants (c_1, c_2, c_3) to heuristic values $(\tilde{c}_1, \tilde{c}_2, \tilde{c}_3)$ in order to make the rare event more frequent under the biased dynamics, and reweight samples with the corresponding likelihood ratio to keep the estimator unbiased. Empirically, the IS strategy leads to a larger number of observed rare events compared to CMC at the same computational budget (See Table 1).

Table 1: Number of observed occurrences of the rare event under Crude Monte Carlo (CMC) and under Importance Sampling (IS), for different sample sizes.

N. simulations	N. observations (CMC)	N. observations (IS)
10^2	0	29
10^3	0	323
10^4	0	3170
10^5	3	31130

To choose the heuristic values, we observed the trajectories obtained with the second set of parameters and initial state (figure 4). From the plots, the main issue is that in the time window $[0, 1]$ too few reactions occur. In particular, we do not see reaction R_2 and R_3 “replacing” reaction R_1 : in these trajectories almost only R_1 fires. Therefore, the main problem is the low number of total firings, not the choice among reaction channels. In SSA, the time increment Δt between two consecutive firings is sampled from an exponential distribution with parameter

$a_0(x) = \sum_{j=1}^3 a_j(x)$. As a consequence, if we increase $a_0(x)$ then the sampled Δt becomes smaller on average, and more reactions are likely to happen within $[0, 1]$. For this reason, in Importance Sampling we choose tilted parameters $\tilde{c}$ in order to increase a_0 and obtain more firings, making it more likely to reach the rare event $\{X_3(T) > 22\}$. However, we should not increase all components of $\tilde{c}$ uniformly. Increasing the rates of reactions that consume S_3 would indeed create more firings, but it would also reduce $X_3(t)$, which is unfavorable for our target event. Hence, $\tilde{c}$ should mainly increase the propensity of the reactions that produce S_3 or keep S_1 in the phenomena (to produce again S_3 later), while keeping the propensity of the last reaction that definitively removes S_3 under control. After some attempts with different $\tilde{c}$ selected according to the above criterion, we pick $(\tilde{c}_1, \tilde{c}_2, \tilde{c}_3) = (0.0025, 0.01, 0.01)$. Three output trajectories with these new parameters are in figure 5.

A reasonably accurate estimate is already obtained with $N = 100$ under Importance Sampling. However, to make a fair comparison with Crude Monte Carlo and to quantify the variance reduction, we focus on the largest budget $N = 10^5$. In this case, the crude estimator is $\hat{Z}_{\text{CMC}, 10^5} = 3 \times 10^{-5}$, while the Importance Sampling estimator is $\hat{Z}_{\text{IS}, 10^5} = 2.604 \times 10^{-5}$. The corresponding estimated variances are

$$\widehat{\text{Var}}(\hat{Z}_{\text{CMC}, 10^5}) = 2.999 \times 10^{-10}, \quad \widehat{\text{Var}}(\hat{Z}_{\text{IS}, 10^5}) = 4.986 \times 10^{-14}.$$

Therefore, Importance Sampling achieves a variance reduction of about four orders of magnitude compared to Crude Monte Carlo at the same sample size.

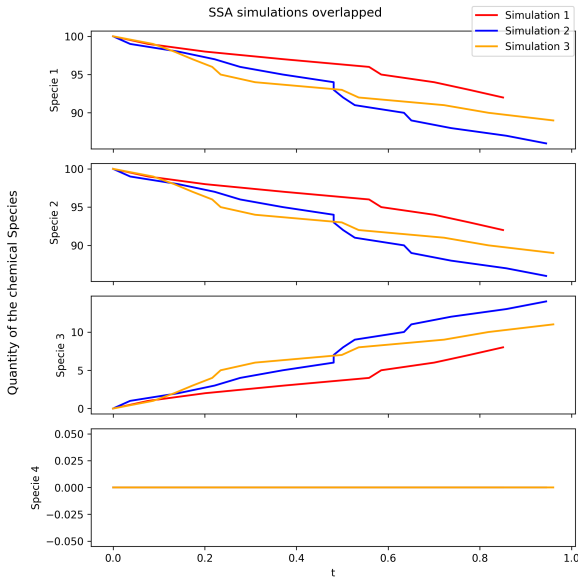


Figure 4: SSA with given parameter c : example trajectories of $X_i(t)$ for $i = 1, \dots, 4$ up to $T = 1$.

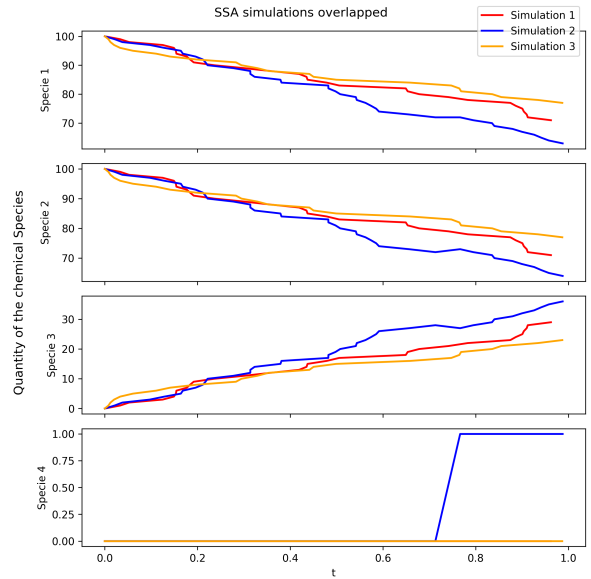


Figure 5: SSA with heuristic parameters $\tilde{c}$ for Importance Sampling: example trajectories of $X_i(t)$ for $i = 1, \dots, 4$ up to $T = 1$.

3 Approximate simulation algorithms

Exact SSA simulation may be expensive when many reactions fire over the interval of interest. We therefore consider approximate schemes based on time discretization.

3.1 Explicit tau-leaping

Fix a time step $\tau > 0$ and assume the rates $a_j(X(t))$ remain constant on $[t, t + \tau)$. The tau-leaping update reads

$$X(t + \tau) = X(t) + \sum_{j=1}^3 \nu_j P_j(a_j(X(t)) \tau), \quad (5)$$

where $P_j(a_j(X(t)) \tau)$ counts the number of times the j -th reaction fired in $[t, t + \tau)$. One of the goals is to show that $p_j \sim P_j(a_j(\mathbf{X}(t)) \tau)$ has a Poisson distribution with parameter $a_j(\mathbf{X}(t)) \tau$.

Let $(p_j(s))_{s \in [t, t + \tau]}$, with $p_j(s)$ the value of p_j at time s .
Let $h \rightarrow 0^+$. We have that

$$\mathbb{P}(p_j(s + h) - p_j(s) = 0) = 1 - a_j(\mathbf{X}(s))h = 1 - a_j(\mathbf{X}(t))h$$

and

$$\mathbb{P}(p_j(s + h) - p_j(s) = 1) = a_j(\mathbf{X}(s))h = a_j(\mathbf{X}(t))h$$

Since this is one of the characterizations of a Poisson process, we can conclude that $(p_j(s))_{s \in [t, t + \tau]}$ acts as a Poisson process with rate $a_j(\mathbf{X}(t))$, so $p_j = p_j(t + \tau) - p_j(t)$ has a Poisson distribution with parameter $a_j(\mathbf{X}(t)) \tau$.

To show the independence of the p_j 's, one can simply observe that each p_j depends only on $a_j(\mathbf{X}(t))$, all of which are kept constant. Therefore the p_j 's just act as independent Poisson processes (each reaction channel behaves like an independent Poisson process).

We simulated the Michaelis–Menten system up to $T = 50$ with the same parameters and initial condition as the SSA experiment and with time step $\tau = 0.2$. Plots are in figure 9.

Qualitatively, trajectories reproduce the same overall trends as SSA (S_1 consumption, S_4 formation, and S_2 - S_3 interplay), while being computationally cheaper per sample path. The approximation introduces a discretization bias, which typically increases with larger τ .

3.1.1 Monte Carlo estimation of $\mathbb{E}[X_4(T)]$ with explicit tau-leaping

Denote by $Z_{\Delta t} = \mathbb{E}[X_4(T)]$ the quantity estimated when using a time discretization (to emphasize the discretization bias). As in section 2.2, we fix the threshold $\varepsilon = 1$ and iteratively increase N to obtain the desired Confidence Interval. With $\tau = 0.2$ and $\alpha = 0.05$, our results are

$$\hat{Z}_{\Delta t} = 295.470, \quad N = 51,$$

with confidence interval

$$\text{CI} = (294.564, 296.376),$$

and estimated variance $\widehat{\text{Var}}(\hat{Z}_{\Delta t}) \approx 0.2136$. Compared to SSA, the confidence intervals are of similar width for the same N , but the point estimate is slightly shifted, consistent with a small time-discretization bias. We expect that decreasing τ the shift also decrease and the estimator gets closer to the value estimated through SSA algorithm. Estimating again with different values of τ we get the results in table 2. To better assess the magnitude of the bias relative to the SSA reference, we use the Z_{SSA} with highest precision, the one obtained for $N = 10^4$ in figure 3. This yields a much tighter confidence interval, allowing us to treat the resulting estimate as a reliable benchmark for comparison with $Z_{\text{TL}, \Delta t}$. Therefore, the reference value obtained is $\hat{Z}_{\text{SSA}} = 296.818$ with $\text{CI} = (296.757, 296.921)$ and $\widehat{\text{Var}}(\hat{Z}_{\text{SSA}}) = 0.00176$.

Interestingly, by chance the point estimate obtained with $N = 10^4$ is exactly the same as the one obtained with the first much smaller sample size of $N = 55$.

From Table 2 we observe that reducing the time step τ (down to half of its initial value) brings the estimator closer to the SSA reference. Consistently, the confidence interval, which initially did not overlap the SSA one, becomes compatible and eventually includes the reference interval. Further decreasing τ does not lead to a clear additional improvement: the point estimate may appear slightly more shifted than in the case $\tau = 0.1$, even though the SSA confidence interval is still contained within the tau-leaping interval. This is likely due to the stopping threshold used to select N , which is not tight enough to let us distinguish the increasingly small differences as τ decreases. Using a smaller threshold would improve the resolution, but at the cost of a substantially larger computational effort. Overall, the expected reduction of the bias as τ decreases is confirmed.

Consider now the behavior of the estimator as N increases, shown in Figure 6. For small sample sizes the estimator exhibits some oscillations, but the values remain fairly close to the SSA reference. As N becomes large, the confidence interval shrinks but the estimator appears to converge to a slightly lower value than Z_{SSA} . This confirms the presence of a discretization bias that does not vanish by increasing N .

Table 2: $Z_{\Delta t} = \mathbb{E}[X_4(T)]$ estimator for different values of τ using explicit Tau Leaping simulation

$Z_{\Delta t}$	C.I.	Variance	N. simulation	τ	$ Z_{\text{SSA}} - Z_{\text{TL}, \Delta t} $
298.540	(297.643, 299.438)	0.2096	37	0.5	1,722
298.653	(297.713, 299.593)	0.2301	49	0.4	1,835
297.352	(296.395, 298.310)	0.2386	34	0.3	0,534
295.470	(294.564, 296.376)	0.2136	51	0.2	1,348
297.083	(296.222, 297.944)	0.1931	48	0.15	0.265
296.719	(295.854, 297.584)	0.1947	57	0.1	0.099
296.661	(295.706, 297.615)	0.2371	59	0.05	0.157
296.417	(295.490, 297.343)	0.223	48	0.01	0.401

3.2 Implicit tau-leaping and Monte Carlo estimation of $\mathbb{E}[X_4(T)]$

To increase stability and allow for potentially larger time steps, an implicit variant can be used. The method first computes an intermediate value y by solving

$$y = X(t) + \tau \sum_{j=1}^3 a_j(y) \nu_j, \quad (6)$$

e.g. with Newton's method, and then performs the stochastic update

$$X(t + \tau) = X(t) + \sum_{j=1}^3 \nu_j P_j(a_j(y) \tau). \quad (7)$$

A representation of some trajectories is in figure 10.

In our experiments, following the same methodology explained in previous sections, under the setting $\tau = 0.2$ and $\alpha = 0.05$, we obtained

$$\hat{Z}_{\Delta t} \approx 295.900, \quad N = 50,$$

with confidence interval

$$\text{CI} = (294.915, 296.885),$$

and estimated variance ≈ 0.2520 . Both explicit and implicit tau-leaping provide similar results for this same value of $\tau = 0.2$ but the implicit one is much slower with our implementation. However, the effectiveness of Implicit Tau Leaping is observable for larger values of τ . Therefore, we run the estimation for different values of τ reported in table 3. As in the explicit case, we expect that the shift with respect the reference value from the SSA simulations decreases as τ decreases.

We emphasize that, for larger values of τ , the Newton solver used to compute the implicit intermediate state does not always converge; for this reason, we implemented two solver variants, using a more robust one when τ is greater than 0.2, at the cost of an additional increase in computational time.

Table 3: $Z_{\Delta t} = \mathbb{E}[X_4(T)]$ estimator for different values of τ using Implicit Tau Leaping simulation

$Z_{\Delta t}$	C.I.	Variance	N. simulation	τ	$ Z_{SSA} - Z_{TL, \Delta t} $
293.419	(292.435, 294.402)	0.2518	74	1.0	3,399
295.935	(294.937, 296.934)	0.2596	62	0.8	0,883
295.911	(294.943, 296.878)	0.2437	56	0.6	0,907
296.145	(295.167, 297.123)	0.2490	62	0.5	0,673
296.304	(295.388, 297.219)	0.2181	56	0.4	0,514
294.877	(293.931, 295.823)	0.2336	65	0.3	1,941
295.900	(294.915, 296.885)	0.2520	50	0.2	0.918

Compared to Explicit Tau-Leaping, Implicit Tau-Leaping already provides estimates that are consistent with the SSA reference even for relatively large step sizes. In fact, for τ in the range $[0.8, 0.4]$ the point estimates are close to Z_{SSA} and the confidence intervals remain compatible with the benchmark, whereas in the explicit case large τ typically leads to a more evident mismatch.

On the other hand, the expected trend “smaller $\tau \Rightarrow$ smaller shift” is not clearly visible. This is mainly due to Monte Carlo noise and to the fact that our stopping criterion produces confidence intervals that are still too wide to resolve small differences between step sizes. The value obtained at $\tau = 0.3$ is an outlier in terms of point estimate, but it is still explainable within the same framework: with a limited number of simulations the estimator can fluctuate noticeably, and the relatively large confidence interval indicates that this deviation is not statistically significant. Overall, the table suggests that the implicit scheme is more robust for large τ , while a finer study of the bias-versus- τ trend would require tighter error thresholds (and thus a higher computational cost).

Figure 7 shows the estimator as a function of N . For small sample sizes it exhibits few oscillations, while for larger N it stabilizes and the confidence interval shrinks. However, even at large N the estimate remains noticeably below the SSA reference value, indicating a persistent shift that is consistent with a non-negligible discretization bias in this setting for any N .

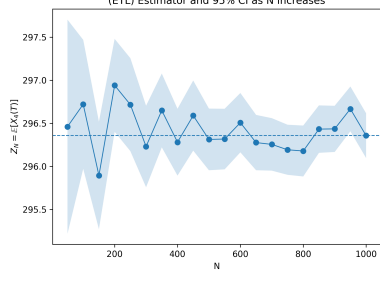


Figure 6: Monte Carlo estimation of $\mathbb{E}[X_4(T)]$ and its C.I. through Explicit Tau Leaping vs N simulated paths.

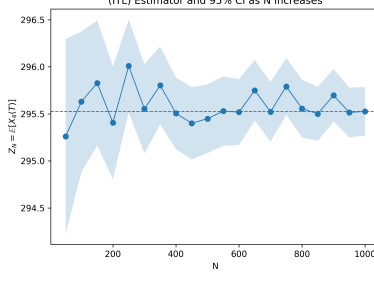


Figure 7: Monte Carlo estimation of $\mathbb{E}[X_4(T)]$ and its C.I. through Implicit Tau Leaping vs N simulated paths.

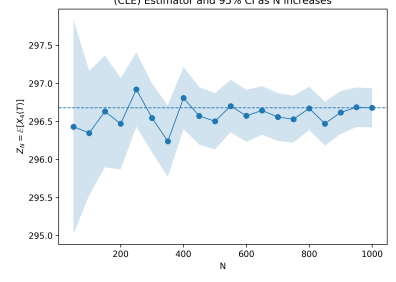


Figure 8: Monte Carlo estimation of $\mathbb{E}[X_4(T)]$ and its C.I. through CLE vs N simulated paths.

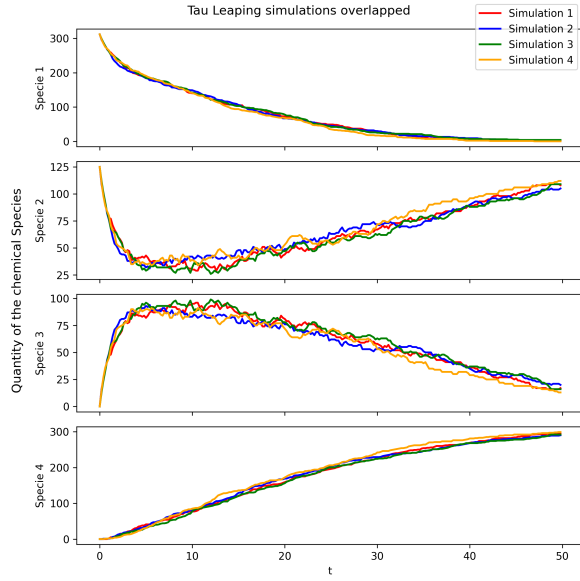


Figure 9: Explicit Tau Leaping: example trajectories of $X_i(t)$ for $i = 1, \dots, 4$ up to $T = 50$.

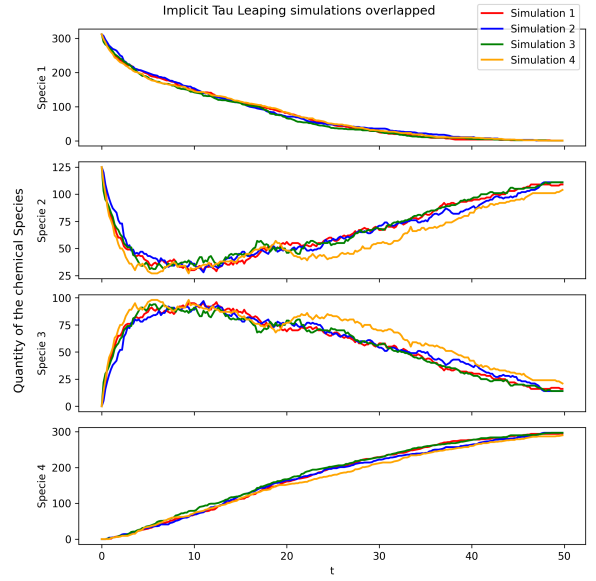


Figure 10: Implicit Tau Leaping: example trajectories of $X_i(t)$ for $i = 1, \dots, 4$ up to $T = 50$.

3.3 Chemical Langevin Equation (CLE)

When all $a_j(X(t))\tau$ are sufficiently large, Poisson increments can be approximated by Normal increments, leading to the Chemical Langevin Equation (CLE)

$$dY(t) = \sum_{j=1}^3 \nu_j a_j(Y(t)) dt + \sum_{j=1}^3 \nu_j \sqrt{a_j(Y(t))} dW_j(t),$$

where W_j are independent Brownian motions and $Y(t) \in \mathbb{R}^4$. Using (2)–(4) and the stoichiometric vectors, the component-wise form is

$$\begin{aligned} dY_1(t) = & -c_1(Y_1(t) \cdot \mathbb{1}_{Y_1(t) \geq 1} \cdot Y_2(t) \cdot \mathbb{1}_{Y_2(t) \geq 1} \cdot \mathbb{1}_{Y_3(t) \geq 0} \cdot \mathbb{1}_{Y_4(t) \geq 0}) \\ & + c_2(\mathbb{1}_{Y_1(t) \geq 0} \cdot \mathbb{1}_{Y_2(t) \geq 0} \cdot Y_3(t) \cdot \mathbb{1}_{Y_3(t) \geq 1} \cdot \mathbb{1}_{Y_4(t) \geq 0}) \\ & - \sqrt{c_1(Y_1(t) \cdot \mathbb{1}_{Y_1(t) \geq 1} \cdot Y_2(t) \cdot \mathbb{1}_{Y_2(t) \geq 1} \cdot \mathbb{1}_{Y_3(t) \geq 0} \cdot \mathbb{1}_{Y_4(t) \geq 0})} dW(t) \end{aligned}$$

$$+ \sqrt{c_2(\mathbb{1}_{Y_1(t) \geq 0} \cdot \mathbb{1}_{Y_2(t) \geq 0} \cdot Y_3(t) \cdot \mathbb{1}_{Y_3(t) \geq 1} \cdot \mathbb{1}_{Y_4(t) \geq 0})} dW^2(t) \quad (8)$$

$$\begin{aligned} dY_2(t) = & -c_1(Y_1(t) \cdot \mathbb{1}_{Y_1(t) \geq 1} \cdot Y_2(t) \cdot \mathbb{1}_{Y_2(t) \geq 1} \cdot \mathbb{1}_{Y_3(t) \geq 0} \cdot \mathbb{1}_{Y_4(t) \geq 1}) \\ & + c_2(\mathbb{1}_{Y_1(t) \geq 0} \cdot \mathbb{1}_{Y_2(t) \geq 0} \cdot Y_3(t) \cdot \mathbb{1}_{Y_3(t) \geq 1} \cdot \mathbb{1}_{Y_4(t) \geq 0}) \\ & + c_3(\mathbb{1}_{Y_1(t) \geq 0} \cdot \mathbb{1}_{Y_2(t) \geq 0} \cdot Y_3(t) \cdot \mathbb{1}_{Y_3(t) \geq 1} \cdot \mathbb{1}_{Y_4(t) \geq 0}) \\ & - \sqrt{c_1(Y_1(t) \cdot \mathbb{1}_{Y_1(t) \geq 1} \cdot Y_2(t) \cdot \mathbb{1}_{Y_2(t) \geq 1} \cdot \mathbb{1}_{Y_3(t) \geq 0} \cdot \mathbb{1}_{Y_4(t) \geq 1})} dW(t) \\ & + \sqrt{c_2(\mathbb{1}_{Y_1(t) \geq 0} \cdot \mathbb{1}_{Y_2(t) \geq 0} \cdot Y_3(t) \cdot \mathbb{1}_{Y_3(t) \geq 1} \cdot \mathbb{1}_{Y_4(t) \geq 0})} dW^2(t) \\ & + \sqrt{c_3(\mathbb{1}_{Y_1(t) \geq 0} \cdot \mathbb{1}_{Y_2(t) \geq 0} \cdot Y_3(t) \cdot \mathbb{1}_{Y_3(t) \geq 1} \cdot \mathbb{1}_{Y_4(t) \geq 0})} dW^3(t) \end{aligned} \quad (9)$$

$$dY_3(t) = -dY_2(t) \quad (10)$$

$$\begin{aligned} dY_4(t) = & c_3(\mathbb{1}_{Y_1(t) \geq 0} \cdot \mathbb{1}_{Y_2(t) \geq 0} \cdot Y_3(t) \cdot \mathbb{1}_{Y_3(t) \geq 1} \cdot \mathbb{1}_{Y_4(t) \geq 0}) \\ & + \sqrt{c_3(\mathbb{1}_{Y_1(t) \geq 0} \cdot \mathbb{1}_{Y_2(t) \geq 0} \cdot Y_3(t) \cdot \mathbb{1}_{Y_3(t) \geq 1} \cdot \mathbb{1}_{Y_4(t) \geq 0})} dW^3(t) \end{aligned} \quad (11)$$

3.3.1 Euler–Maruyama discretization

We simulate (8)–(11) using Euler–Maruyama with step $\tau = 0.2$ and initial condition $Y(0) = X(0)$ as in section 2.2. Since $Y(t)$ is real-valued and can become negative due to Gaussian noise, a practical fix is to apply a projection such as taking component-wise absolute values or truncating negative values to zero. We opted for the first fix.

From figure 11, we qualitatively observe that CLE captures the macroscopic trend of the SRN similarly to tau-leaping and SSA, while typically being faster per time step. However, accuracy depends strongly on the validity of the Gaussian approximation (large propensities) and on the chosen handling of negativity.

3.3.2 Monte Carlo estimation of $\mathbb{E}[X_4(T)]$ with CLE and E-M discretization

Here we estimate again the value of $\mathbb{E}[X_4(T)]$ but using Chemical Langevin Equation with Euler–Maruyama discretization. The initial setting is the same of the previous sections, with $\tau = 0.2$ and $\alpha = 0.05$. The results are:

$$\hat{Z}_{\Delta t} \approx 296.648, \quad N = 46,$$

with confidence interval

$$\text{CI} = (295.657, 297.638),$$

and estimated variance ≈ 0.2552 . These values are very close to the ones obtained through SSA simulations but with significantly less computational cost (Indeed, in our implementation, it takes 12.3s compared to 80.5s for SSA). To see the behavior of the estimator with respect τ , we collect the data in table 4 similarly to the other methods.

From Table 4 we observe that the CLE–E–M estimator becomes progressively closer to the SSA reference as the time step τ decreases. In particular, for $\tau \leq 0.4$ the absolute difference from Z_{SSA} is already below 0.3, and it further decreases to 0.036 for $\tau = 0.1$. For larger steps ($\tau = 0.8$ and $\tau = 1.0$) the discrepancy is more evident, indicating that the discretization error dominates in this regime. Overall, these results are consistent with the expected reduction of the time-discretization bias when refining the step size, while the confidence intervals remain of comparable width across τ due to similar Monte Carlo variability and sample sizes determined by the fixed threshold. Moreover, in Figure 8 we observe the trend of the estimator as a function

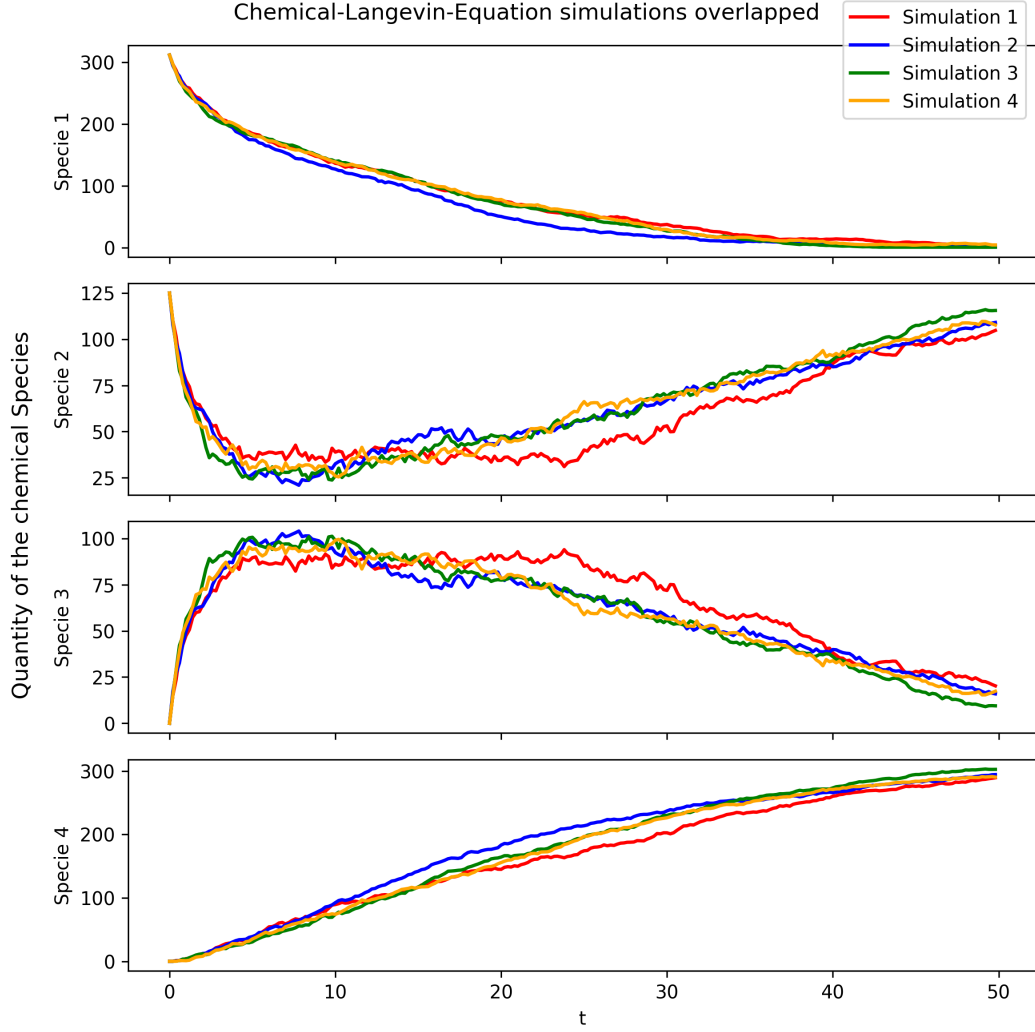


Figure 11: CLE: example trajectories of $X_i(t)$ for $i = 1, \dots, 4$ up to $T = 50$.

of N . The estimate quickly stabilizes close to the SSA reference value, and the confidence interval shrinks as expected. Under the considered setting and implementation, this is the most favorable behavior among the approximated methods.

3.4 Comparison between CLE and Tau Leaping algorithms

In order to compare these two approximated algorithms we fix a priori the sample size $N = 1000$ and the time step $\tau = 0.2$. We recall here the initial setting for the paths simulation: $X_1(0) = 312$, $X_2(0) = 125$, $X_3(0) = 0$, $X_4(0) = 0$ and $c_1 = 0.0017$, $c_2 = 10^{-4}$, $c_3 = 0.1$. The results are shown in table 5.

From the table we observe that, for the same computational budget ($N = 1000$) and the same time step ($\tau = 0.2$), both methods achieve confidence intervals of comparable width and very similar estimated variances. However, the CLE-E-M estimator is noticeably closer to the SSA reference value, with an absolute difference of only 0.004, while explicit tau-leaping shows a larger deviation (0.389), suggesting a stronger bias at this step size for this specific setting. In terms of efficiency, CLE-E-M is also slightly faster in our implementation (6.1s vs 7.8s). Overall, for $\tau = 0.2$ the Gaussian-based approximation provides a better accuracy-cost trade-off than explicit tau-leaping, at least for the estimation of $\mathbb{E}[X_4(T)]$ in the considered setting.

Table 4: $Z_{\Delta t} = \mathbb{E}[X_4(T)]$ estimator for different values of τ using CLE with EM discretization

$Z_{\Delta t}$	C.I.	Variance	N. simulation	τ	$ Z_{SSA} - Z_{TL, \Delta t} $
299.072	(298.101, 300.043)	0.2453	62	1.0	2,254
298.602	(297.716, 299.488)	0.2045	52	0.8	1.784
297.461	(296.505, 298.417)	0.2379	50	0.5	0,643
297.117	(296.182, 298.052)	0.2274	52	0.4	0,299
297.074	(296.135, 298.014)	0.2299	60	0.3	0.256
296.648	(295.657, 297.638)	0.2552	46	0.2	0.170
296.782	(295.821, 297.744)	0.2410	56	0.1	0.036

Table 5: $Z_{\Delta t} = \mathbb{E}[X_4(T)]$ estimator for different approximated algorithms: CLE vs Explicit Tau Leaping. Comparison with fixed sample size $N = 1000$ and the time step $\tau = 0.2$

	$Z_{\Delta t}$	C.I.	Variance	running time (s)	$ Z_{SSA} - Z_{X, \tau} $
CLE-ME	296.814	(296.549, 297.079)	0.0183	6.1	0.004
Explicit TL	296.429	(296.171, 296.687)	0.0173	7.8	0.389

4 Discussion and Conclusion

The SSA provides an exact simulation of the SRN dynamics, but it can become computationally expensive when many reactions fire over the time horizon. Tau-leaping methods address this cost by grouping multiple reaction firings within a fixed time step: the explicit scheme is simple and efficient, but it introduces a time-discretization bias that typically increases with τ and may lead to instability for large steps. The implicit tau-leaping variant improves robustness and allows the use of larger step sizes while preserving the integer-valued nature of the process, at the price of solving a nonlinear system at each step. The Chemical Langevin Equation (CLE) offers a Gaussian approximation that can be computationally attractive when propensities are sufficiently large, but it may require additional corrections to handle negative values and can lose accuracy when the approximation is not valid.

In our Michaelis–Menten experiments with $T = 50$ and $\tau = 0.2$, the approximated methods produced confidence intervals of comparable width to SSA with a similar number of Monte Carlo samples, while showing small shifts in the point estimates consistent with discretization and approximation errors. The implicit scheme proved more reliable for larger time steps, whereas the CLE provided a competitive accuracy–cost trade-off in the considered regime. For the rare-event probability estimation, crude Monte Carlo was ineffective at moderate sample sizes, while Importance Sampling drastically increased the number of observed rare events and achieved a substantial variance reduction, making the estimation feasible within practical computational budgets.

Overall, SSA remains the reference method for accuracy, while tau-leaping and CLE provide effective alternatives when computational efficiency is required and step sizes are chosen appropriately. For rare-event problems, variance reduction techniques such as Importance Sampling are essential to obtain meaningful estimates.

A Code

Here is the code for the functions of the simulation methods, Crude Monte Carlo and importance sampling. You can also find these functions in the code located in the .zip file.

```
1  ## general setting of the system
2  #NR = 3          # Number of Reactions
3  #NS = 4          # Number of Species
4  alpha = np.array([[1,1,0,0],[0,0,1,0],[0,0,1,0]]) # values from the
   physical system
5  beta = np.array([[0,0,1,0],[1,1,0,0],[0,1,0,1]]) # values from the
   physical system
6  ##
7
8  ## setting for part 2.1
9  c = np.array([0.0017, 10**-4, 0.1]) # constant values
   given in proj. descr.
10 X = np.array([312, 125, 0, 0], dtype=int) # initial states of
   the four species
11 t = 0 # initial time
12 T = 50 # final time
13 ##
14
15 ### functions defintion for the stochastic simulation of the
   MichaelisMenten system
16
17 def propensity(X,c,alpha):
18     """
19     input:
20         X      : current states (vector of dim=num_species of INTEGERS
   )
21         c      : (vector of constants with dim=num_reactions)
22         alpha  : (coeff of the left side of the system).
23     output:
24         a      : propensity function (vector of dim=num_reactions)
25     explanation:
26         compute the formula given in the project description for
   integers X_i
27     """
28
29
30     a = np.array(c)
31     for j in range(0,len(c)):
32         for i in range(0,len(X)):
33             if X[i]>=alpha[j][i]:
34                 a[j] = a[j]*m.perm(X[i],alpha[j][i])
35             else:
36                 a[j] = 0 #if there is not enough Species for some
   reaction
37
38     return a
39
40 def sim_step_SSA(X,t,T,c,alpha,beta):
41     """
42     input:
43         X      : current states (vector of dim=num_species)
```

```

44     t      : initial time
45     T      : final time (last t is > T)
46     c      : (vector of constants with dim=num_reactions)
47     alpha   : (coeff of the left side of the system)
48     beta    : (coeff of the right side of the system)
49 output:
50     X_next  : states after one step(vector of dim=num_reactions)
51     t_next  : time at which next reaction happen
52 explanation:
53     do a step of the simulation accordingly to Algorithm 1 of proj.
54     descr.
55
56     """
57
58     a = propensity(X,c,alpha)
59     a_sum = sum(a)
60     if a_sum == 0:
61         ### comment the print when running multiple simulations for
62         estimations ###
63         #print(f"At this this state, no reaction can happen.
64         Equilibrium reached at time {t}")
65         return X, T
66     pmf = a/a_sum #pmf of j-th reaction to happen
67
68     I = np.random.choice(np.arange(len(c)),p=pmf) # sample of the
69     next reaction to happen
70     tau = np.random.exponential(1/a_sum) # sample the time for
71     the next reaction
72     nu_I = beta[I]-alpha[I] # stoichiometric vector
73     associated to reaction I
74
75     X_next = X + nu_I
76     t_next = t + tau
77
78     return X_next, t_next
79
80 # general simulation function
81 def simulation(sim_step,X,t,T,*step_args,max_iter=10**5):
82     """
83     input:
84         sim_step      : function of the simulation step, depends on
85                         numerical algorithm to use
86                         sim_step HAS TO take in input (X,t,T, ...)
87         X             : initial states (vector of dim=num_species)
88         t             : initial time
89         T             : approx final time (last t is > T)
90         step_args     : all the other ordered inputs needed for sim_step
91                         except for X,t,T
92         max_iter      : max number of iteration of sim_step
93     output:
94         X_evo         : evolution of all the states (matrix of dim =
95                         num_time_steps * num_reactions),
96         t_evo         : time steps (vector dim=num_time_steps)
97     explanation:
98         run a simulation of the reactions in according with the
99         algorithm implemented in the
100        simulation step. Use of "while" loop because time step is not
101        always constant so the
102        number of steps is not known a priori. N.B. X,t,T are not in *

```

```

        step_args because they
        are used also for the external loop.
    """
    t_current = t
    X_current = X
    t_evo = [t]
    X_evo = X
    iter_count = 0
    while t_current < T and iter_count < max_iter:
        X_current, t_current = sim_step(X_current, t_current, T, *
            step_args)
        t_evo.append(t_current)
        X_evo = np.vstack([X_evo, X_current])
        iter_count += 1

    if iter_count == max_iter and t_current < T:
        raise RuntimeError("error in simulation loop: max iteration
            reached before final time")

    return X_evo, t_evo

###
## Generic Crude Monte Carlo function useful for successive tasks

def CMC(sig_lev, qoi, N, simulate, *sim_args ):
    """
    input:
        sig_lev : significance level (alpha) for CI
        qoi      : quantity of interest to be estimated (function of
            one simulation result (X,t))
        N       : population (number of simulations)
        simulate : simulation function to compute data for estimation
        sim_args : all positional arguments needed for simulation
            function
    output:
        Z       : estimation of the quantity of interest
        var_est  : Variance of the estimator
        CI       : Confidence Interval
        obs      : number of observation of the qoi (Only for true/
            false qoi)
    explanation:
        compute the value of the estimator for the quantity of interest
            taken from a generic kind of
        simulation with its confidence interval wr to the significance
            level
    """
    samples = []

    for k in range(N):
        X_evo, t_evo = simulate(*sim_args)
        samples.append(qoi(X_evo, t_evo))

    samples = np.array(samples)
    obs = sum(samples)
    Z = samples.mean()
    se = samples.std(ddof=1) / np.sqrt(N)      # standard error = sqrt(

```

```

    Var(est))
141 var_est = se**2                                # Var(est)
142
143 q = st.norm.ppf(1 - sig_lev/2)                  # alpha-quantile of normal
    distribution
144 CI = (Z - q*se, Z + q*se)
145
146 return Z, var_est, CI, obs
147
148 ##
149
150 ## Section 2.1 goal 5 -- construction of an Importance Sampling
    function
151
152 def weight_func_SSA(X_evo, t_evo, T, c, c_tilde, alpha, beta):
153     """
154     input:
155         X_evo      : evolution of the species of one simulation runned wr
                        to c_tilde
156         t_evo      : evolution of time of one simulation runned wr to
                        c_tilde
157         T          : final time
158         c          : parameter of the propensity of the original phenomena
159         c_tilde    : modified parameter of the ppropensity for the
                        dominating distr. for IS
160         alpha      : (coeff of the left side of the system)
161         beta       : (coeff of the right side of the system)
162     output:
163         w          : weight function for IS evaluated for the input sim
164     explanation:
165         compute the value of the weight function for the importance
            sampling through
166         the analytical expression derived in the case of a !SSA
            algortihm! for a certain
167         simulation. The expression is derived computing the likelihood
            functions of a general
168         chain produced by the standard distribution (with parameter c)
            and by the dominating
169         one (with parameter c_tilde), evaluating both of them in the
            chain produced by the
170         dominating distribution and making the ratio. Therefore, X_evo
            and t_evo have to be
171         produced in the c_tilde case.
172     """
173     t = t_evo[:-1] + [T]
174     X = np.vstack((X_evo[:-1], X_evo[-2]))
175
176     w = 0
177     for i in range(0, len(t)-1):
178         a_c_tilde = propensity(X[i], c_tilde, alpha)
179         a_c_tilde_sum = sum(a_c_tilde)
180         a_c = propensity(X[i], c, alpha)
181         a_c_sum = sum(a_c)
182         for J in range(0, len(c)):
183             if np.array_equal(X[i+1] - X[i], beta[J] - alpha[J]):
184                 break
185
186         w += (np.log(a_c[J]) - np.log(a_c_tilde[J])) * (i < (len(t)-2)) - (

```

```

        a_c_sum-a_c_tilde_sum)*(t[i+1]-t[i])
187
188     return np.exp(w)
189
190 def IS(c_tilde, sig_lev, qoi, N, simulate, *sim_args, index_c = -3):
191     """
192     input:
193         c_tilde    : c parameters for the dominating distribution
194         sig_lev    : significance level (alpha) for CI
195         qoi        : quantity of interest to be estimated (function of
196                      one simulation result (X,t))
197         N          : population (number of simulations)
198         simulate   : simulation function to compute data for estimation
199         sim_args   : all positional arguments needed for simulation
200                      function
201     output:
202         Z_IS       : estimation of the quantity of interest with
203                      Importance Sampling
204         var_Z_IS   : Variance of the estimator with Importance Sampling
205         CI_IS      : Confidence Interval
206         obs        : number of observations of qoi (Only for True/False
207                      qoi)
208     explanation:
209         compute the value of the estimator for the quantity of interest
210         taken from a generic kind of
211         simulation with its confidence interval wr to the significance
212         level through Importance Sampling
213         based on g (dominating distribution) computed with c_tilde
214         instead of c.
215         N.B. we are assuming that alpha and beta are always the last
216         two elements of sim_args.
217     """
218
219     sim_args_tilde = (*sim_args[0:index_c], c_tilde, *sim_args[index_c
220                        +1:])
221     c = sim_args[index_c]
222     T = sim_args[3]
223     alpha = sim_args[-2]
224     beta = sim_args[-1]
225
226     samples = []
227     obs_vector = []
228
229     for k in range(N):
230         X_evo, t_evo = simulate(*sim_args_tilde)
231         w = weight_func_SSA(X_evo, t_evo, T, c, c_tilde, alpha, beta)
232         obs_vector.append(qoi(X_evo, t_evo))
233         samples.append(qoi(X_evo, t_evo)*w)
234
235     samples = np.array(samples)
236     obs = sum(obs_vector)
237     Z_IS = samples.mean()
238     se = samples.std(ddof=1) / np.sqrt(N)      # standard error = sqrt(
239         Var(est))
240     var_Z_IS = se**2                          # Var(est)
241
242     q = st.norm.ppf(1 - sig_lev/2)            # alpha-quantile of normal
243         distribution

```

```

233     CI_IS = (Z_IS - q*se, Z_IS + q*se)
234
235     return Z_IS, var_Z_IS, CI_IS, obs
236
237 ##
238
239 ## Section 3 - Tau Leaping algorithm (2)
240
241 def sim_step_TL(X,t,T,tau,c,alpha,beta):
242     """
243     input:
244         X          : current states (vector of dim=num_species)
245         t          : initial time
246         T          : final time (last t is > T)
247         tau        : time step
248         c          : (vector of constants with dim=num_reactions)
249         alpha      : (coeff of the left side of the system)
250         beta       : (coeff of the right side of the system)
251     output:
252         X_next     : states after one step(vector of dim=num_reactions)
253         t_next     : time at which next reaction happen
254     explanation:
255         do a step of the simulation accordingly to Algorithm 2 (Tau
256         Leaping) of proj. descr.
257     """
258
259     a = propensity(X,c,alpha)
260     a_sum = sum(a)
261     if a_sum == 0:
262         ### comment the print when running multiple simulations for
263         estimations ###
264         #print(f"At this this state, no reaction can happen.
265         Equilibrium reached at time {t}")
266         return X, T
267
268     p = np.random.poisson(a*tau)
269     nu = beta-alpha
270     delta_X = p @ nu
271
272     # take max wr 0 because doing only one big jump for all reactions
273     we could get negative states
274     X_next = np.maximum(X+delta_X, 0)
275     t_next = t + tau
276
277     return X_next, t_next
278
279 ##
280
281 ## functions definitions for the Implicit Tau Leaping Algorithm
282 def continuous_propensity(X,c,alpha):
283     """
284     input:
285         X          : current states (vector of floats of dim=num_species)
286         c          : (vector of constants with dim=num_reactions)
287         alpha      : (coeff of the left side of the system).
288     output:
289         a          : propensity function (vector of dim=num_reactions)
290     explanation:

```

```

287         compute the formula given in the project description but for
288         float inputs. Needed for
289         the Newton method since we need a differentiable function to
290         apply it.
291
292         """
293
294         a = np.array(c)
295         for j in range(0,len(c)):
296             for i in range(0,len(X)):
297                 if X[i]>=alpha[j][i]:
298                     for r in range(0,alpha[j][i]):
299                         a[j] = a[j]*(X[i]-r)
300                 else:
301                     a[j] = 0      #if there is not enough Species for some
302                                 reaction
303
304         return a
305
306 def function_for_newton(y,X,tau,c,alpha,beta):
307     """
308     input:
309         y                : state X(t)+tau*a(y)@nu (vector dim=
310                        num_species)
311         X                : current states (vector of dim=num_species)
312         tau              : time step
313         c                : (vector of constants with dim=num_reactions)
314         alpha            : (coeff of the left side of the system)
315         beta             : (coeff of the right side of the system)
316     output:
317         y-X-tau*drift    : function we need to make 0 trough newton
318                        method to find y
319     explanation:
320         compute the non linear function to be solved through newton
321         method for the Implicit
322         Tau Leaping algorithm
323     """
324     nu = beta-alpha
325     a = continuous_propensity(y,c,alpha)
326     drift = a@nu
327     return y - X - tau*drift
328
329 """
330 This commented function below was the first function for newton method
331 but for larger tau it didn't converge.
332 To our understanding of the problems, the root function could make
333 trials with negative values and the
334 initial try might be too far from the actual solution. Moreover, bigger
335 tau might imply almost singular Jacobian
336 which produces bigger newton steps that could diverge.
337 Therefore, we wrote the other uncommented function with a different
338 solver, with the constraint of non negative
339 components and a better initialization.
340
341 def solve_implicit_y(X,tau,c,alpha,beta):
342     y0 = np.array(X)
343     sol = so.root(function_for_newton, y0, args=(X,tau,c,alpha,beta),
344                  method="hybr")

```

```

334     if not sol.success :
335         raise RuntimeError("Newton/root do not converge: " + sol.
336                             message)
337     return sol.x
338
339 """
340 # another version for numerical stability BUT MUCH SLOWER - use the
341 other for tau<=0.2
342 def solve_implicit_y(X,tau,c,alpha,beta):
343     """
344     input:
345         X          : current state at time t (vector of length =
346                     num_species).
347         tau         : time step size.
348         c          : reaction rate constants (vector of length =
349                     num_reactions).
350         alpha       : stoichiometric coefficients of reactants (matrix
351                     num_reactions x num_species).
352         beta        : stoichiometric coefficients of products (matrix
353                     num_reactions x num_species).
354     output:
355         y          : implicit intermediate state (vector of floats of
356                     length = num_species)
357                     solving  $y - X - \tau * (a(y) @ (beta - alpha)) = 0$ .
358     explanation:
359         This function computes the intermediate value y required by the
360         Implicit Tau-Leaping method.
361         The value y is defined implicitly by a nonlinear system, which
362         we solve numerically.
363
364         Compared to a standard root-finding call, we use a more robust
365         least-squares solver with
366         non-negativity bounds ( $y \geq 0$ ) to avoid non-physical
367         intermediate states that often cause
368         convergence issues for large tau. We also use a better initial
369         guess y0 obtained from an
370         explicit drift step,  $y0 = X + \tau * (a(X) @ nu)$ , and then clipped
371         to be non-negative. This
372         combination typically improves numerical stability and reduces
373         failures of the solver.
374
375     """
376     X = np.asarray(X, dtype=float)
377     nu = beta - alpha
378     # better initial guess: explicit drift step (mean-field)
379     aX = continuous_propensity(np.maximum(X, 0.0), c, alpha)
380     y0 = np.maximum(X + tau * (aX @ nu), 0.0)
381     sol = so.least_squares(
382         function_for_newton,
383         y0,
384         bounds=(0.0, np.inf),      # enforce non-negativity
385         args=(X, tau, c, alpha, beta),
386         max_nfev=2000
387     )
388     if not sol.success :
389         raise RuntimeError("Newton/root do not converge: " + sol.
390                             message)
391     return sol.x

```

```

377
378 def sim_step_ITL(X,t,T,tau,c,alpha,beta):
379     """
380     input:
381         X          : current states (vector of dim=num_species)
382         t          : initial time
383         T          : final time (last t is > T)
384         tau        : time step
385         c          : (vector of constants with dim=num_reactions)
386         alpha      : (coeff of the left side of the system)
387         beta       : (coeff of the right side of the system)
388     output:
389         X_next     : states after one step(vector of dim=num_reactions)
390         t_next     : time at which next reaction happen
391     explanation:
392         do a step of the simulation accordingly to Algorithm of
393             Implicit Tau Leaping of proj. descr.
394             section 3.1.1 goal 4.
395         N.B. Even if we compute "a" with continuous_propensity (so as a
396             vector of float numbers),
397             the final vector state will be of integers since a is used only
398             as an argument of the poisson
399             distribution, from which we sample integers.
400     """
401     y = solve_implicit_y(X,tau,c,alpha,beta)
402     a = continuous_propensity(y,c,alpha)
403     a_sum = sum(a)
404     if a_sum == 0:
405         ### comment the print when running multiple simulations for
406         estimations ###
407         #print(f"At this this state, no reaction can happen.
408         Equilibrium reached at time {t}")
409         return X, T
410
411     p = np.random.poisson(a*tau)
412     nu = beta-alpha
413     delta_X = p @ nu
414     delta_X = np.array(delta_X, dtype=int)
415
416     # take max wr 0 because doing only one big jump for all reactions
417     we could get negative states
418     X_next = np.maximum(X+delta_X, 0)
419     t_next = t + tau
420
421     return X_next, t_next
422
423 ##
424
425 ## function definition for the Euler-Maruyama scheme
426
427 def sim_step_EM(X,t,T,tau,c,alpha,beta):
428     """
429     input:
430         X          : current states (vector of dim=num_species)
431         t          : initial time
432         T          : final time (last t is > T)
433         tau        : time step
434         c          : (vector of constants with dim=num_reactions)

```

```

429         alpha    : (coeff of the left side of the system)
430         beta     : (coeff of the right side of the system)
431     output:
432         X_next    : states after one step(vector of dim=num_reactions)
433         t_next    : time at which next reaction happen
434     explanation:
435         do a step of the simulation accordingly to Euler-Maruyama
436             scheme
437     """
438     a = continuous_propensity(X,c,alpha)
439     a_sum = sum(a)
440     if a_sum == 0:
441         ### comment the print when running multiple simulations for
442         estimations ###
443         #print(f"At this this state, no reaction can happen.
444         Equilibrium reached at time {t}")
445         return X, T
446
447     z_vec = np.random.normal(0,1,size=len(a))
448     nu = beta-alpha
449     delta_X = tau*(a @ nu) + np.sqrt(tau)*((z_vec * np.sqrt(a)) @ nu )
450
451     # take max wr 0 because doing only one big jump for all reactions
452     we could get negative states
453     X_next = np.abs(X+delta_X)
454     t_next = t + tau
455
456     return X_next, t_next
457
458 ##

```